

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-016: Show / Hide Password Visibility Toggle in the Login Module

Field	Details
Change Request ID	CR-016
Change Request Title	Show / Hide Password Visibility Toggle in the Login Module
Student Name	MH Anush
Student ID	JFICT2416
Implementation Status	Completed
Date	9 October 2026

1. Change Request Summary

1.1 Business Requirement

Users and administrators previously entered passwords into a masked input field without an option to view or verify entered text. As passwords often contain complex combinations of numbers, symbols, and casing, users frequently made typographical errors without realizing it. The lack of visibility increased failed login attempts and repeated entries. The change was required to provide an intuitive client-side password toggle allowing users to view or mask credentials before submitting.

1.2 Existing Behaviour

Before implementation:

- `LoginPage.jsx` rendered `LoginFormSection.jsx` with username and password fields.
- The password input was statically configured with `type="password"`.
- Password input characters were masked unconditionally with bullets.
- No visibility toggle button or eye icon existed to reveal entered text.
- Existing authentication sent credentials via `POST /api/auth/login`.
- `StudentFormModal.jsx` did not provide password visibility toggle functionality in student account management.

1.3 Requested Change

A client-side show/hide password toggle was required on the login password input (and student form modal).

The implementation must:

- Toggle password input type dynamically between "password" and "text".
- Default to masked/hidden state upon component mount and page load.
- Display intuitive eye and eye-slash visual indicator icons.
- Include accessible `aria-label` and `aria-pressed` attributes.
- Ensure toggle button uses `type="button"` to prevent accidental form submission.
- Adjust layout in `login.css` and `dashboard.css` so password text does not overlap the icon.
- Support keyboard activation via Space and Enter keys.
- Preserve existing login validation and error alert banners.
- Pass entered credentials accurately to `login(username, password)`.
- Maintain existing administrator and student authentication and navigation flows.

2. Implementation

2.1 Implementation Summary

The CR-016 requirements were implemented in the frontend login module and extended to the student form modal. A local React state (`showPassword`) was introduced in `LoginFormSection.jsx` to manage visibility state. The password input type dynamically switches between "password" and "text". Interactive SVG icons (Eye and EyeOff) from `lucide-react` were integrated with accessible `aria-label` and `aria-pressed` attributes.

Input styling was updated with a dedicated `password-input-wrapper` and 44px right padding to ensure entered characters never collide with or hide behind the toggle button. In addition, the password toggle feature was extended to `StudentFormModal.jsx`, enabling administrators to verify passwords when adding or updating student records.

2.2 Affected Files

File	Change	Purpose
<code>frontend/src/components/LoginFormSection.jsx</code>	Modified	Implemented show/hide toggle state, dynamic input type switching, accessible SVG buttons, and keyboard support.
<code>frontend/src/styles/login.css</code>	Modified	Added <code>.password-input-wrapper</code> and <code>.password-toggle-btn</code> styles with 44px padding-right to prevent text overlap.
<code>frontend/src/components/StudentFormModal.jsx</code>	Modified	Extended show/hide password toggle to Add Student and Edit/Update Student password inputs.

File	Change	Purpose
frontend/src/styles/dashboard.css	Modified	Added styling for password wrapper and toggle button in the student modal card.
backend/src/controllers/studentController.js	Modified	Updated editStudent controller to hash and update student password if provided.
backend/src/utls/validation.js	Modified	Updated validation utility to allow optional password update during student edits.

Additional files: None / [List any additional files if modified]

2.3 Technical Changes

Visibility State Management

A local state was implemented in LoginFormSection.jsx:

- `const [showPassword, setShowPassword] = useState(false);`
- When false (default): `type="password"`, `aria-label="Show password"`, Eye icon displayed.
- When true: `type="text"`, `aria-label="Hide password"`, EyeOff icon displayed.

Layout and Spacing

A flex container with relative positioning was implemented:

- The input receives `padding-right: 44px` so password text remains fully legible.
- The toggle button is positioned absolutely on the right with vertical center alignment.
- Button uses `type="button"` to ensure activating it does not submit the login form.

Accessibility and Security

- Full keyboard operability: Tab key navigates to the button, and Space/Enter activates the toggle.
- Screen reader support with dynamic `aria-label` and `aria-pressed` attributes.
- Passwords are never stored in `localStorage`, `sessionStorage`, or browser cookies.
- Default masked state on load protects against shoulder-surfing risks.

2.4 Implementation Logic

1. Component mounts with `showPassword` initialized to false (masked state).
2. User types password credentials into the masked input field.
3. User clicks the toggle button or presses Space / Enter while focused.
4. State toggles to true; input type instantly switches to "text".
5. SVG icon dynamically switches from Eye to EyeOff.
6. Password text remains completely visible without clipping or overlapping the button.
7. User clicks toggle again; state toggles to false; input type reverts to "password".
8. Form submission passes credentials unaltered to `login(username, password)`.

9. Existing JWT token storage and role-based redirect (/dashboard or /student-dashboard) occur seamlessly.

2.5 Error and Data Handling

The existing authentication error handling remains completely intact:

- Empty username or password submission triggers existing required-field validation banners.
- Invalid credentials return HTTP 401 and display the standard error alert banner.
- Submitting valid credentials in either masked or visible state completes login successfully.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through manual frontend testing and browser verification.

Testing Types:

- Functional testing
- Positive testing
- Validation testing
- UI testing
- Accessibility / keyboard testing
- Regression testing
- Authentication verification

Environment: Development

Browser: [Chrome / Edge / Firefox]

Test Role: Administrator & Student

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Default masked state on load	Password input has type="password", characters masked with bullets, Eye icon visible	Input is type="password", bullets displayed, Eye icon shown	Pass
TC-02	Click toggle button to reveal password	Input type changes to "text", plaintext characters revealed, EyeOff icon displayed	Input switches to "text", text revealed, EyeOff shown	Pass
TC-03	Click toggle button to hide password	Input type reverts to "password", characters masked with bullets, Eye icon displayed	Input reverts to "password", characters masked, Eye shown	Pass
TC-04	Input long password string	Password text does not collide with or hide under toggle icon due to right padding	Text clearly legible with 44px right padding	Pass

Test ID	Test Case	Expected Result	Actual Result	Status
TC-05	Click toggle does not submit form	Activating toggle button does not trigger login submission or error banners	Form does not submit; type="button" prevents submission	Pass
TC-06	Keyboard navigation and activation	Button is focusable via Tab and toggles state via Space and Enter keys	Button receives focus and toggles with Space and Enter	Pass
TC-07	Submit valid credentials in revealed state	Valid credentials submitted while password is in text mode log in successfully	Login succeeds, redirects to dashboard	Pass
TC-08	Submit valid credentials in masked state	Valid credentials submitted while password is masked log in successfully	Login succeeds, redirects to dashboard	Pass
TC-09	Submit invalid credentials with toggle active	Invalid credentials display error alert banner without breaking toggle	Error banner displayed correctly; toggle continues working	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	9
Passed	9
Failed	0
Pass Rate	100%

Overall Test Result: Pass / Partially Passed / Failed

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Users can toggle password visibility between hidden and visible.	Functional testing (TC-02, TC-03)	Met
AC-002	Default state of the password field on load is masked.	Initial load verification (TC-01)	Met
AC-003	Toggle icon updates dynamically to match visibility state.	Dynamic icon test (TC-02, TC-03)	Met
AC-004	Activating the toggle does not trigger login form submission.	Form button type check (TC-05)	Met
AC-005	Toggle button is keyboard accessible and includes aria-label.	Accessibility verification (TC-06)	Met
AC-006	Password text remains legible and is not obscured by the toggle icon.	UI layout & padding inspection (TC-04)	Met
AC-007	Valid credentials submitted in either state log in successfully.	Authentication testing (TC-07, TC-08)	Met
AC-008	Existing validation errors and role-based redirects continue to work.	Regression verification (TC-07, TC-08, TC-09)	Met

5. Evidence

A maximum of two relevant screenshots should be attached.

Evidence 01 – Password Visibility Toggle (Masked State)

Related: AC-001, AC-002, AC-003, AC-005, AC-006

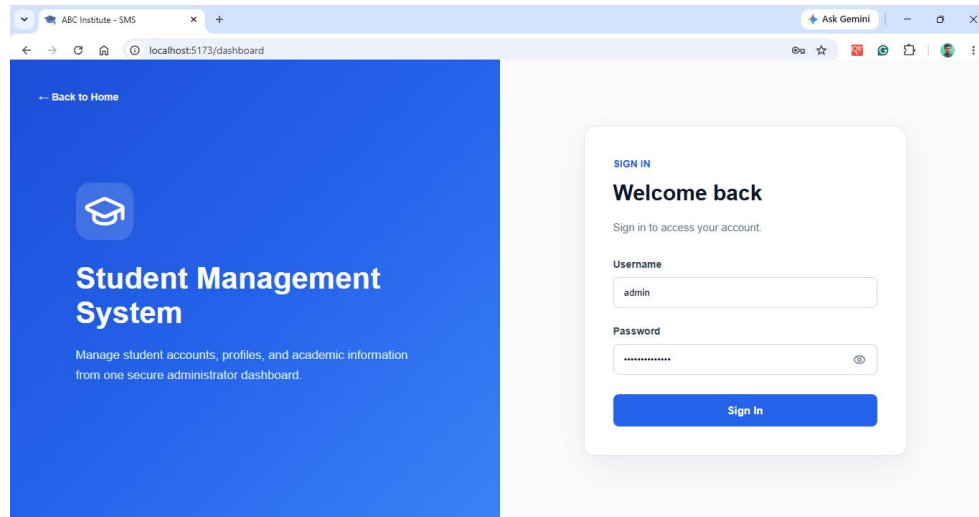


Figure 1: Login module showing masked password field with default bullet masking and eye visibility toggle button.

Evidence 02 – Password Visibility Toggle (Revealed State & Authentication)

Related: AC-001, AC-003, AC-004, AC-007, AC-008

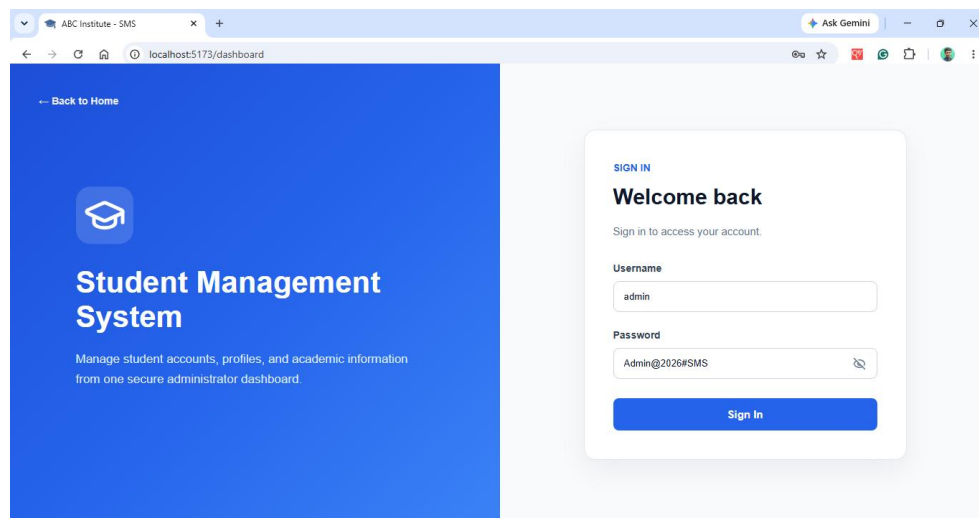


Figure 2: Login module showing revealed plaintext password with eye-slash toggle icon and seamless credential submission.

6. Final Result

Item	Result
Requirements	Met / Partially Met / Not Met
Implementation	Completed / Partially Completed
Testing	Passed / Partially Passed / Failed
Acceptance Criteria	All Met / Partially Met / Not Met
Existing CRUD / Auth	Verified / Not Verified
Final Status	Accepted / Pending / Rejected

6.1 Final Summary

CR-016 was implemented as a client-side password visibility toggle enhancement for the Student Management System login module and student form management. The implementation allows users and administrators to toggle passwords between masked characters and plaintext view, reducing mistyped credential errors while preserving strict security protocols. The toggle button is accessible via keyboard and screen readers, incorporates standard SVG eye / eye-slash visual indicators, and includes layout padding preventing character overlap. Existing authentication flows, JWT token issuance, error alert banners, and role-based redirects remain fully functional. The implemented functionality was tested against all defined acceptance criteria with a 100% pass rate.

6.2 Issues / Limitations

[State any remaining issues or limitations.]

If none:

No significant issues identified.

Student Name: MH Anush

Student ID: JFICT2416

Date: 9 October 2026

Signature: _____

~ END OF REPORT ~